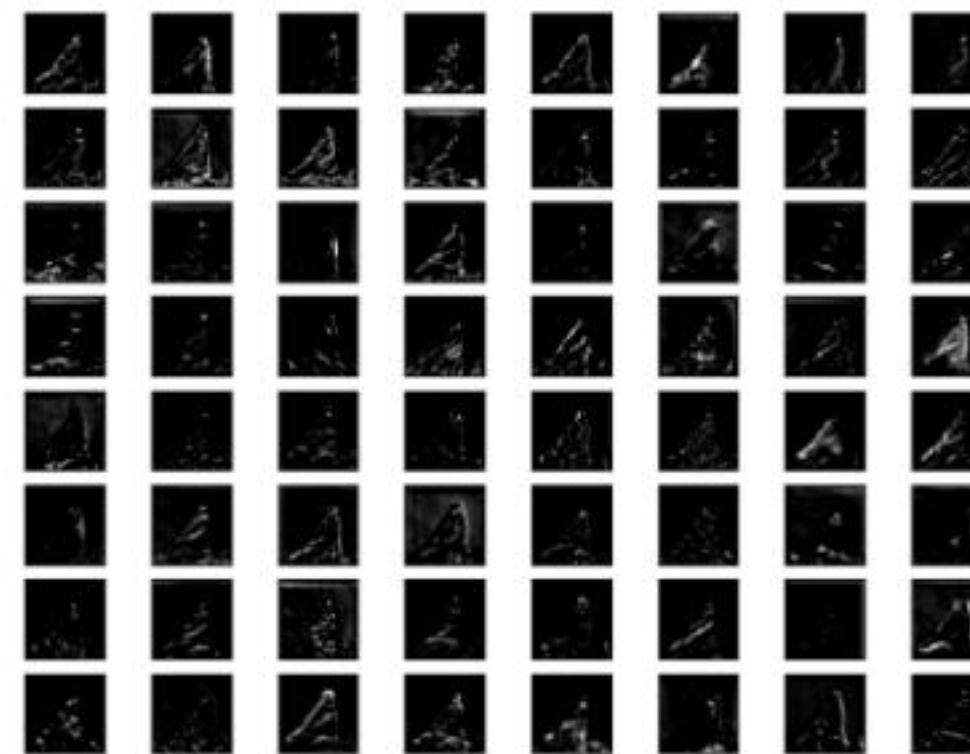


Padding, Stride, Receptive Fields and Pooling

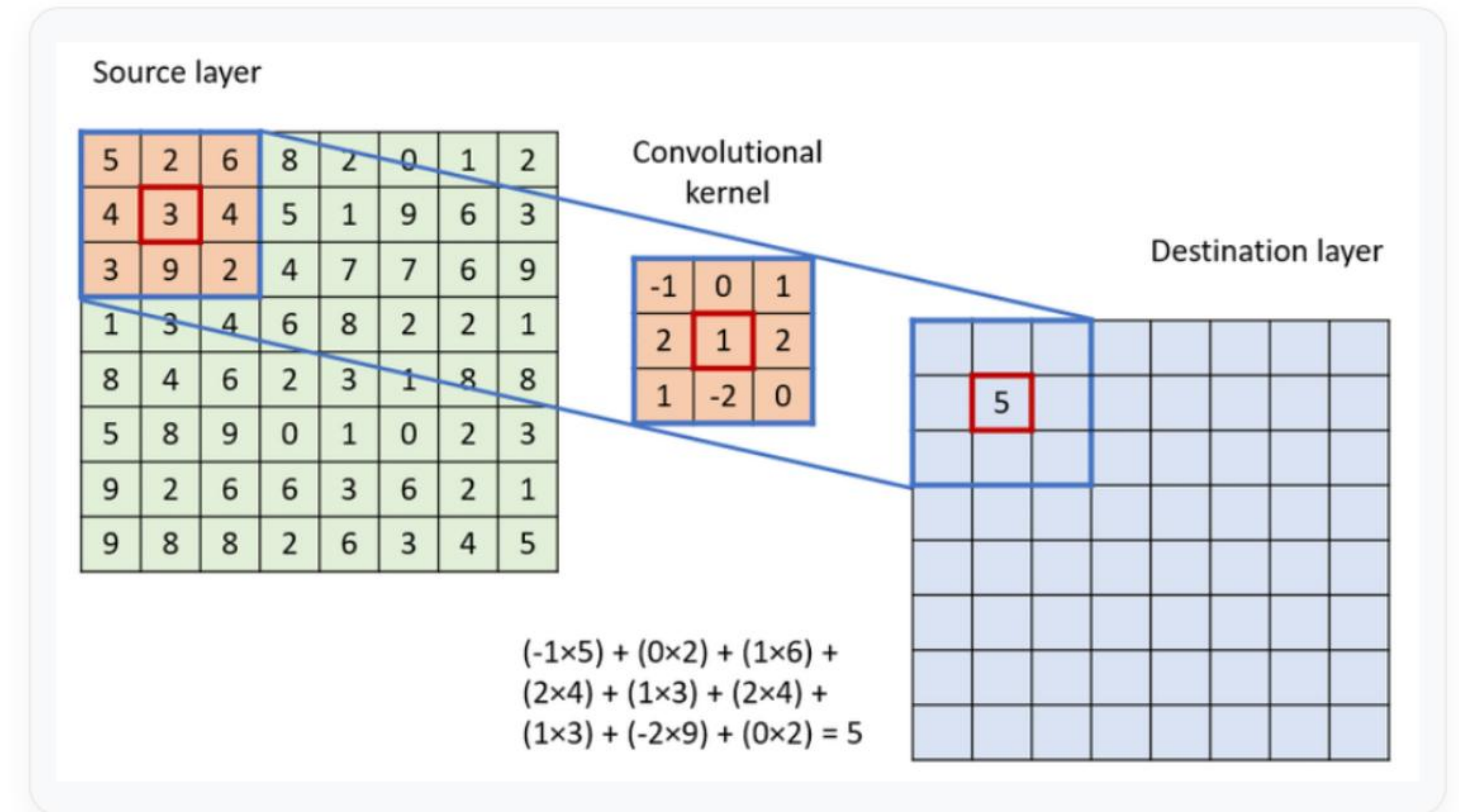
Building Efficient Convolutional Neural Networks



Learning Outcomes

By the end of this lecture, you should be able to:

- Explain how **padding** preserves spatial dimensions.
- Understand how **stride** controls kernel movement.
- Compute **output feature map dimensions** mathematically.
- Define and visualize a **receptive field**.
- Explain how **pooling layers** reduce dimensionality.



Lecture Roadmap

The journey from raw pixels to dense feature representations.



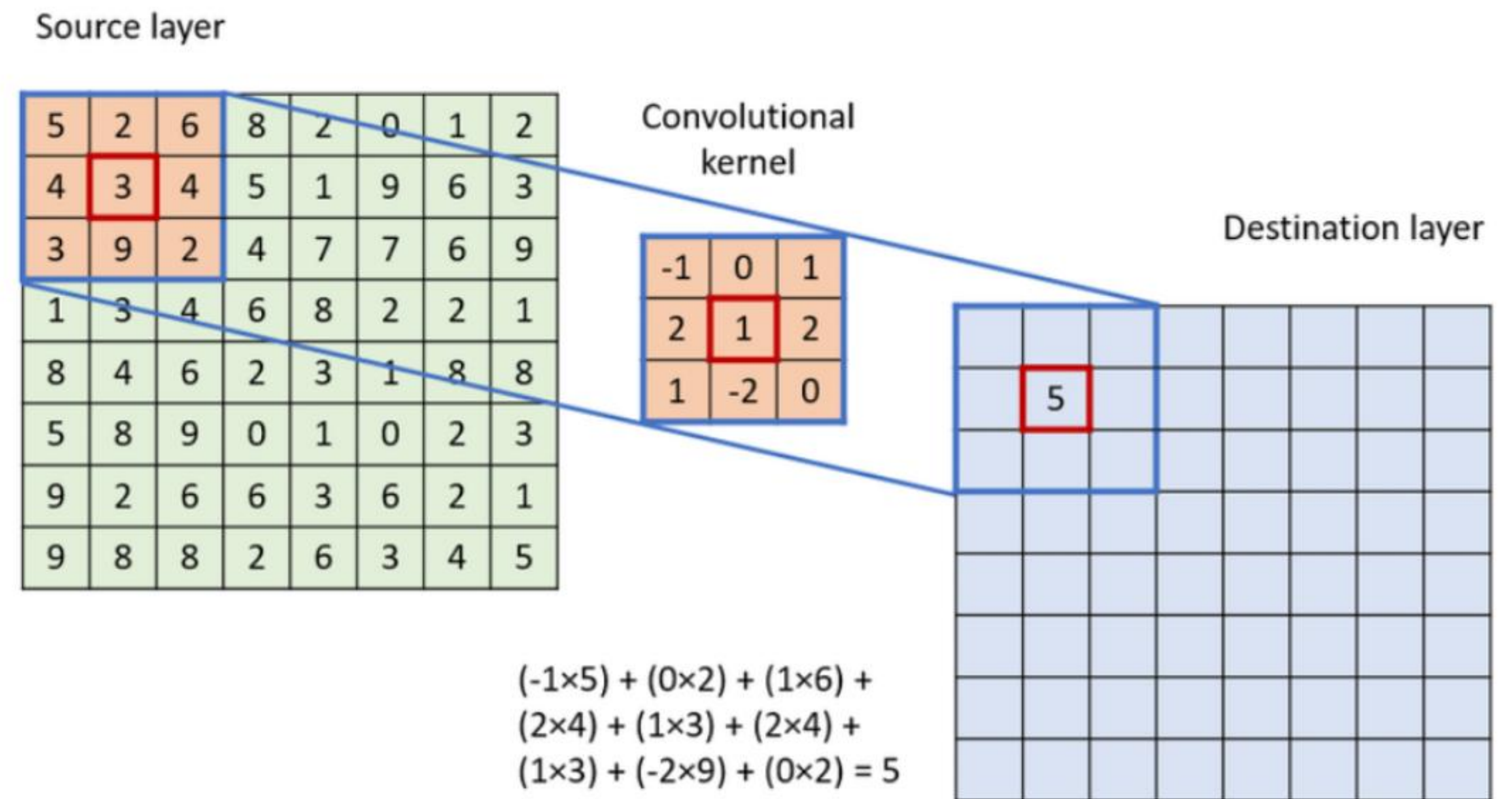
Quick Review: The Convolution Operation

In our last lecture, we learned how a kernel (filter) slides over an image to extract features.

- **Input Image:** Raw pixel values.
- **Kernel:** Small matrix of learnable weights.
- **Feature Map:** Output indicating feature location.

CLASSROOM QUESTION

What happens to the spatial dimensions of the feature map when the kernel reaches the absolute boundary of the image?



The Boundary Problem: Why Padding?

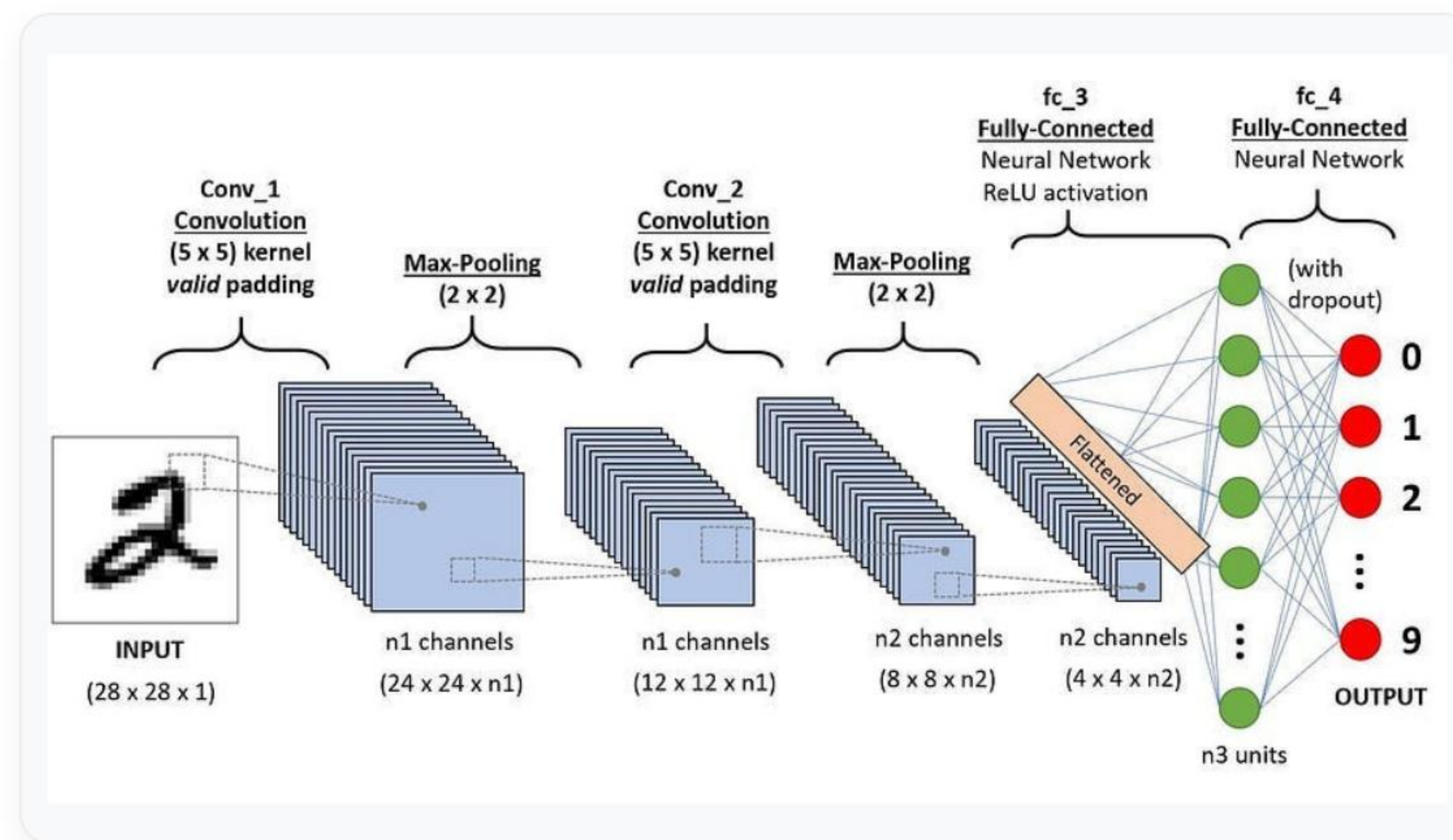
The Shrinking Effect

When applying a 3×3 kernel to a 5×5 image without modifications, the output shrinks to a 3×3 feature map.

- Pixels on the edges are visited **less frequently** than pixels in the center.
- Critical edge information is **lost**.
- Deep networks would shrink the image to nothing rapidly.



Analogy: Padding is like adding a blank picture frame around a photo so edge details aren't covered by matting.



Types of Padding Configurations



Valid Padding

Also known as **No Padding**.

- The kernel only operates where it entirely fits within the image.
- Output spatial dimensions are strictly smaller than the input.



Same Padding

Also known as **Zero Padding**.

- Zeros are added symmetrically around the image border.
- Output spatial dimensions match the input dimensions (when Stride = 1).

DID YOU KNOW?

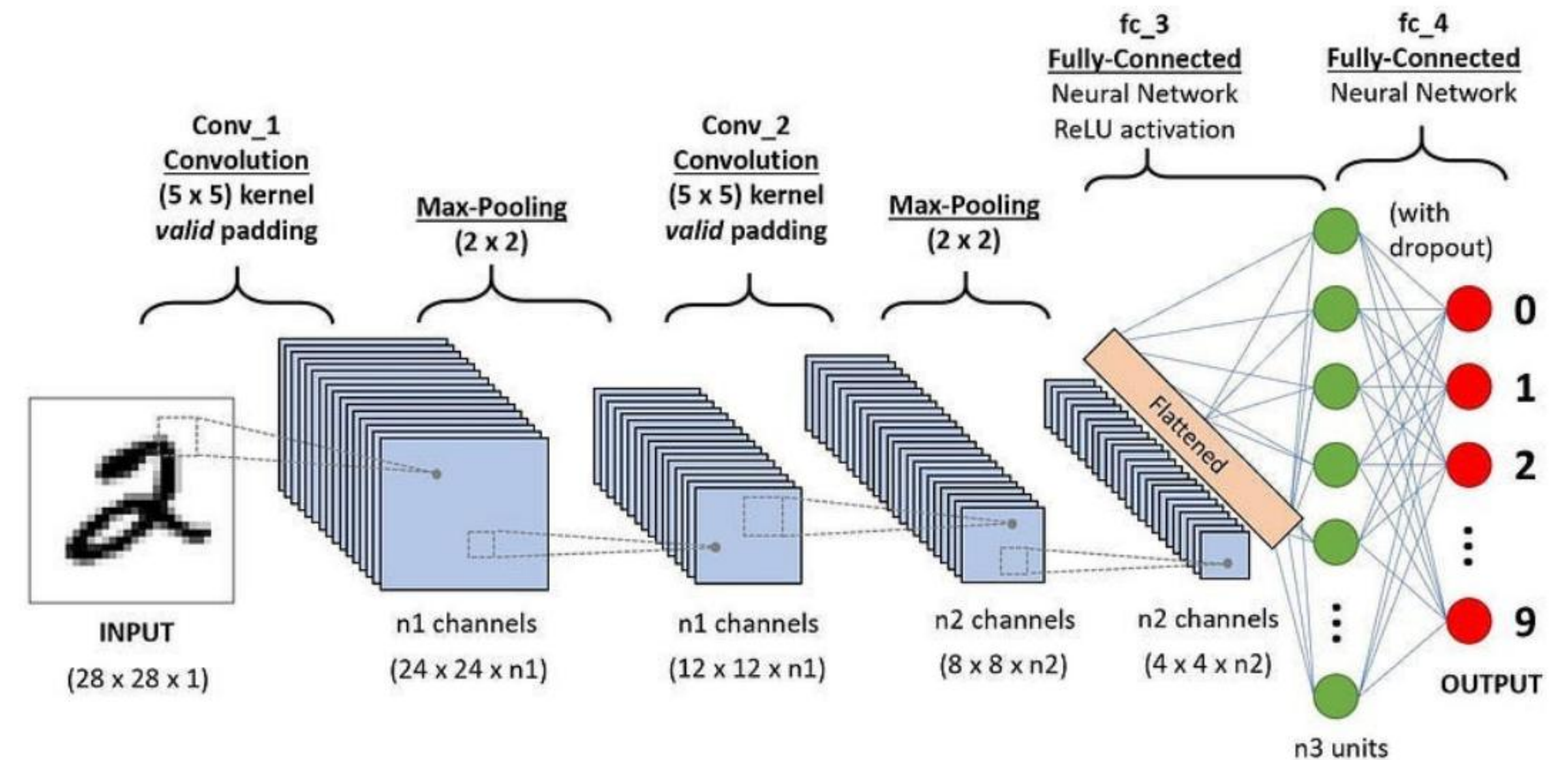
While Zero Padding is standard, researchers also use **Reflection Padding** (mirroring edge pixels) in Generative Adversarial Networks (GANs) to prevent border artifacts.

Padding Example in Action

Visualizing Same Padding ($P = 1$)

Let's take our 5×5 image and apply a padding of 1 on all sides.

- The padded image becomes 7×7 .
- Applying a 3×3 kernel allows the center of the kernel to reach the exact edge of the original 5×5 image.
- The resulting feature map remains 5×5 .



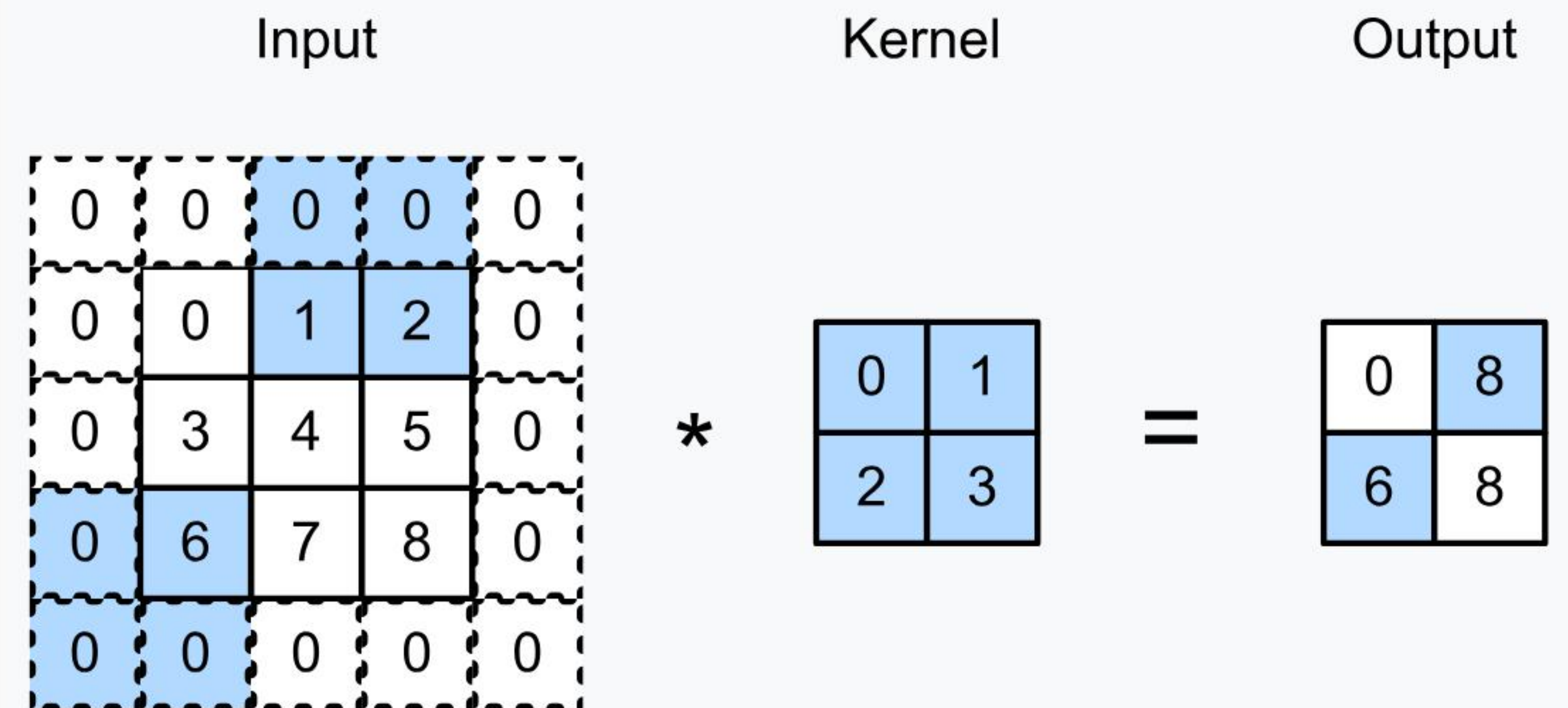
Controlling Movement: What is Stride?

Stride (S) defines the step size the kernel takes as it slides across the input image.

- **Stride = 1:** Moves 1 pixel at a time. Dense feature extraction.
- **Stride = 2:** Skips 1 pixel. Output size is roughly halved.



Analogy: Stride is like walking across floor tiles. You can step on every single tile (Stride 1) or take larger steps, skipping tiles (Stride 2+).



Stride Comparison & Efficiency

Stride Setting	Feature Map Size	Computational Cost	Primary Use Case
S = 1	Dense (Large)	High (Many FLOPs)	Early layers, preserving fine details.
S = 2	Reduced by ~50%	Low (Fewer FLOPs)	Downsampling, expanding receptive field.
S = 3+	Aggressively Small	Very Low	Rare, used only in specific tasks or 1D sequences.

DID YOU KNOW?

Modern architectures like ResNet often replace traditional Pooling layers entirely with **Strided Convolutions (Stride=2)** to let the network *learn* how to downsample optimally.

The Master Equation: Output Size Calculation

To engineer CNNs effectively, we must calculate the exact spatial dimension of the feature map after Convolution, Padding, and Stride.

$$O_{\text{size}} = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

- **W** = Input Width / Height
- **K** = Kernel Size
- **P** = Padding
- **S** = Stride

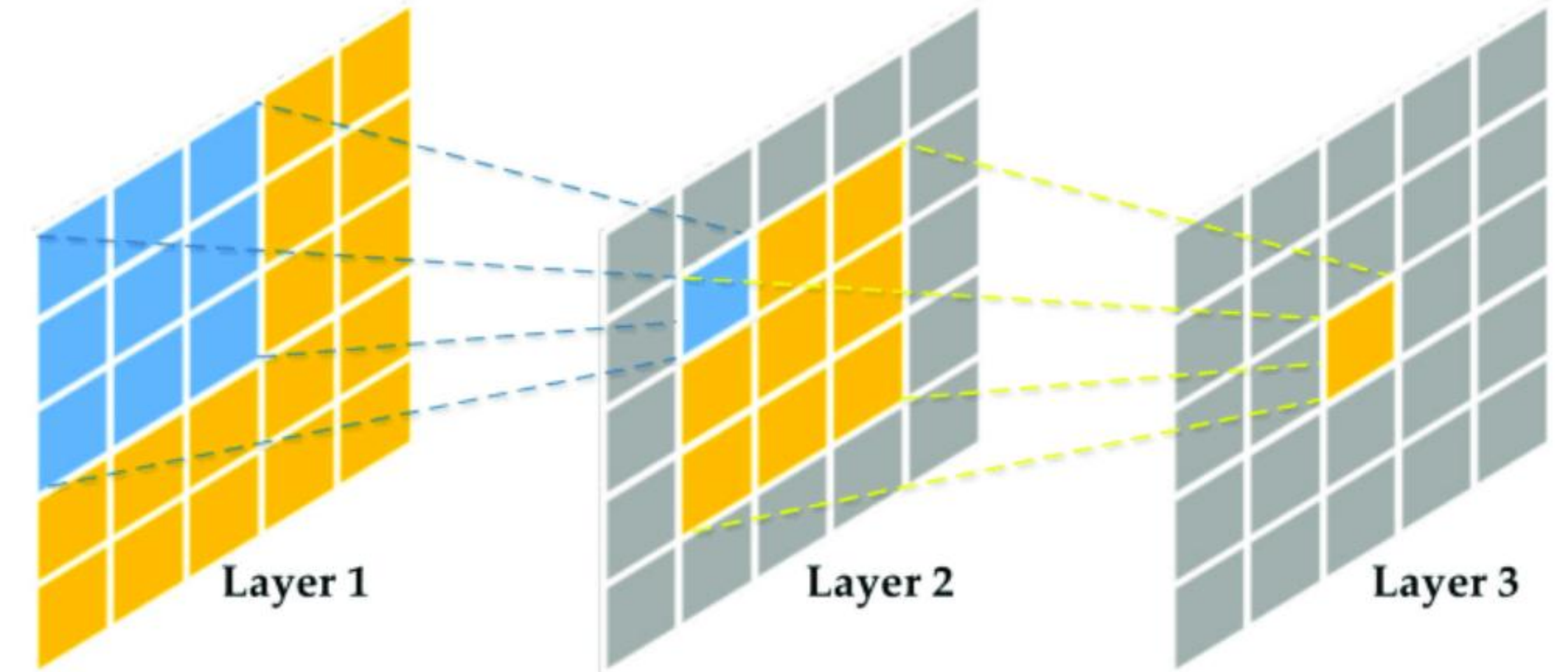
What is a Receptive Field?

The **Receptive Field** is the specific region in the original input image that influences a single neuron in a feature map.

- A single pixel in the first feature map represents a $K \times K$ region of the original image (e.g., 3×3).
- It defines what the network "sees" at any given layer.



Analogy: Looking through a window. The farther back you step (deeper in the network), the wider the view you have of the scene outside.



Hierarchical Learning: Growing Fields

As we stack convolutional layers, the receptive field grows. This is why deep networks can understand complex, global structures.

Layer 1

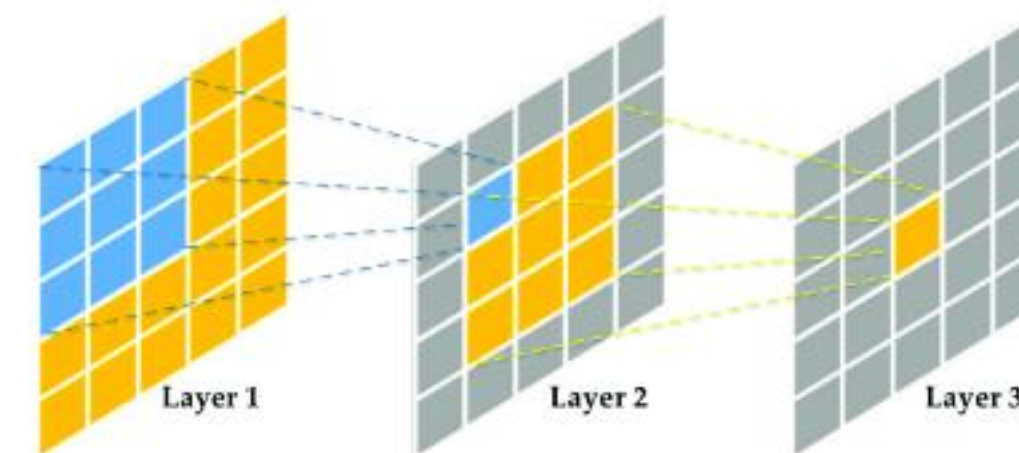
Field: Small (e.g., 3×3)
Learns: Edges, corners.

Layer 3

Field: Medium
Learns: Textures, simple shapes.

Deep Layers

Field: Almost Entire Image
Learns: Complex Objects.



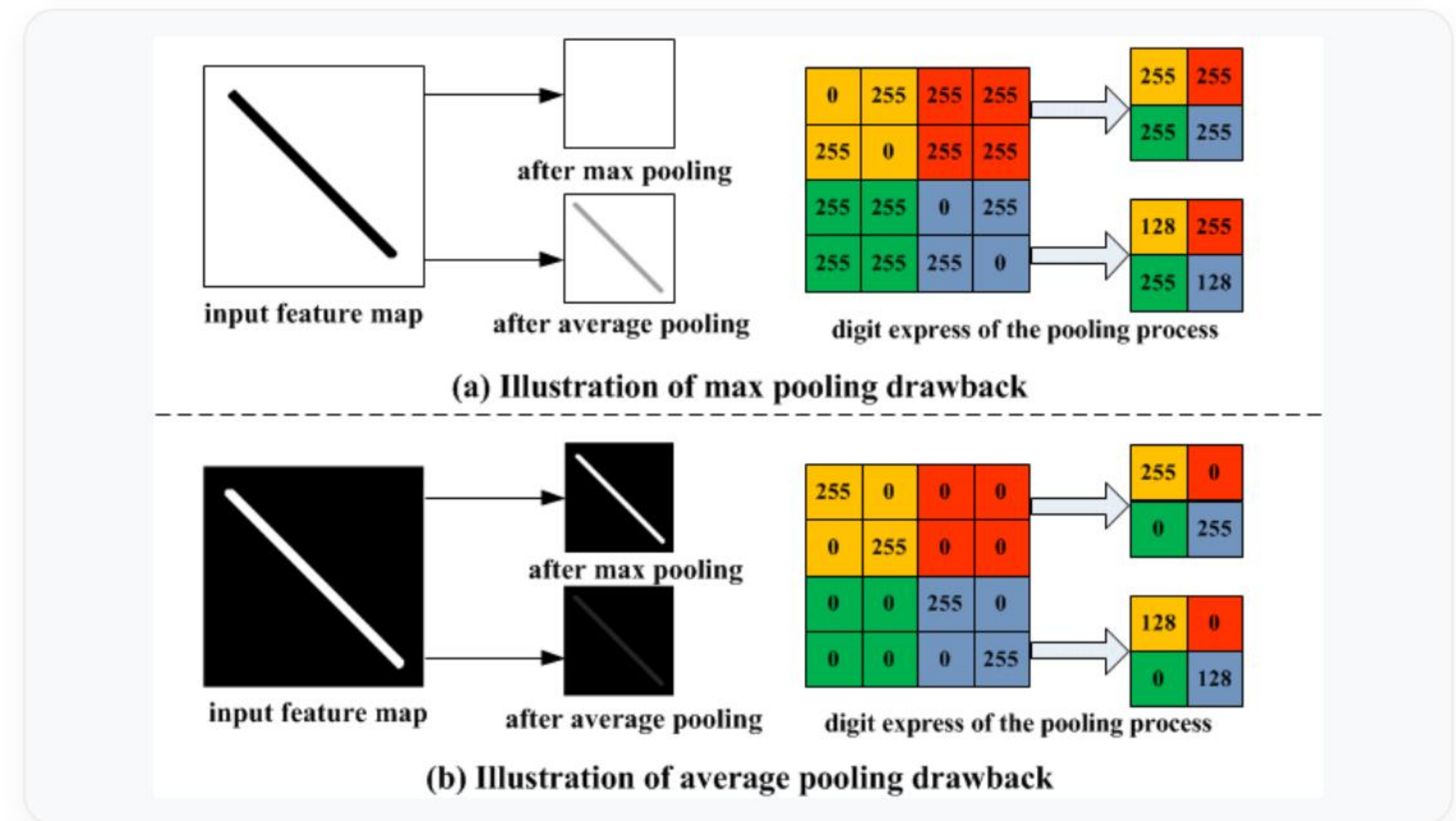
Dimensionality Reduction: Pooling

The Memory Problem

High-resolution images create massive feature maps. Storing and processing these requires enormous memory and compute power.

The Solution: Pooling

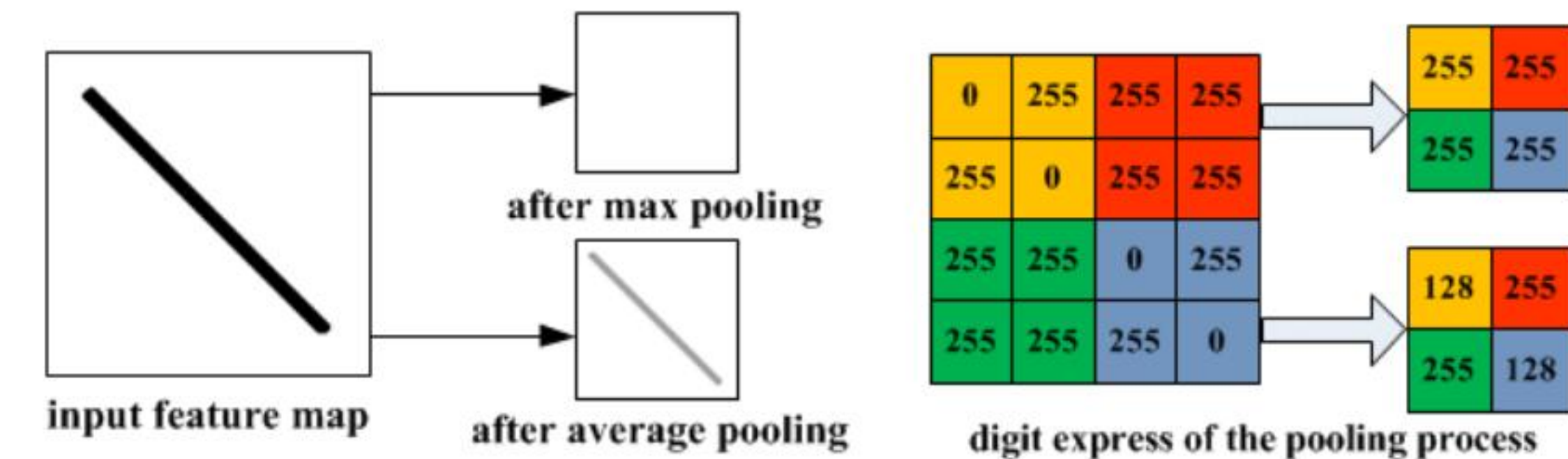
- Summarizes the feature map.
- Reduces spatial dimensions (Width, Height).
- Maintains the depth (Number of Channels).
- Makes network invariant to small translations.



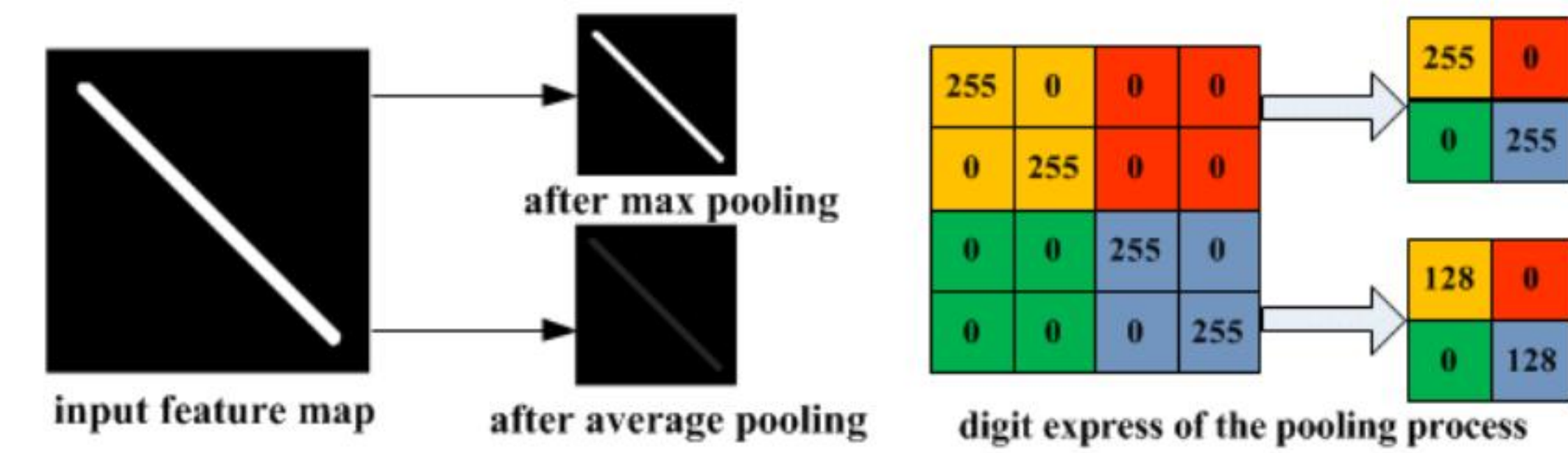
Max Pooling Operations

Max Pooling slides a window (usually 2×2 with Stride 2) and selects only the maximum value within that window.

- **Feature Preservation:** Keeps highly activated features (e.g., the sharpest edge).
- **Noise Suppression:** Ignores lower activations.
- **Non-learnable:** No weights to update!



(a) Illustration of max pooling drawback



(b) Illustration of average pooling drawback

DID YOU KNOW?

Early groundbreaking CNNs like AlexNet and VGG heavily relied on Max Pooling. It is generally the preferred pooling method for computer vision tasks today.

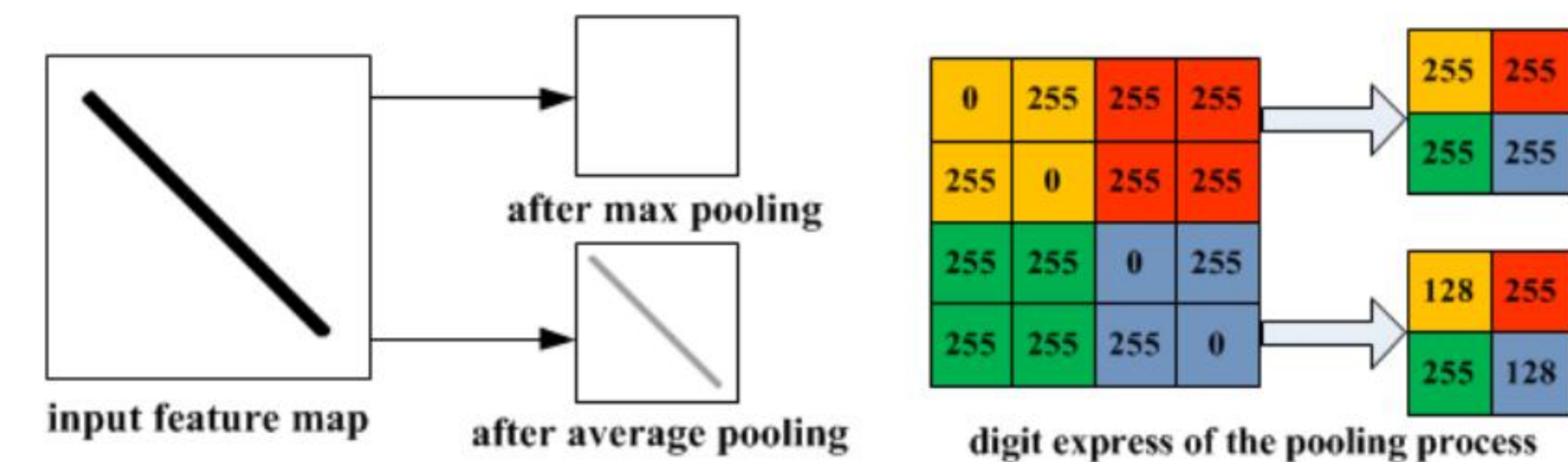
Average Pooling Operations

Average Pooling computes the arithmetic mean of all values within the pooling window.

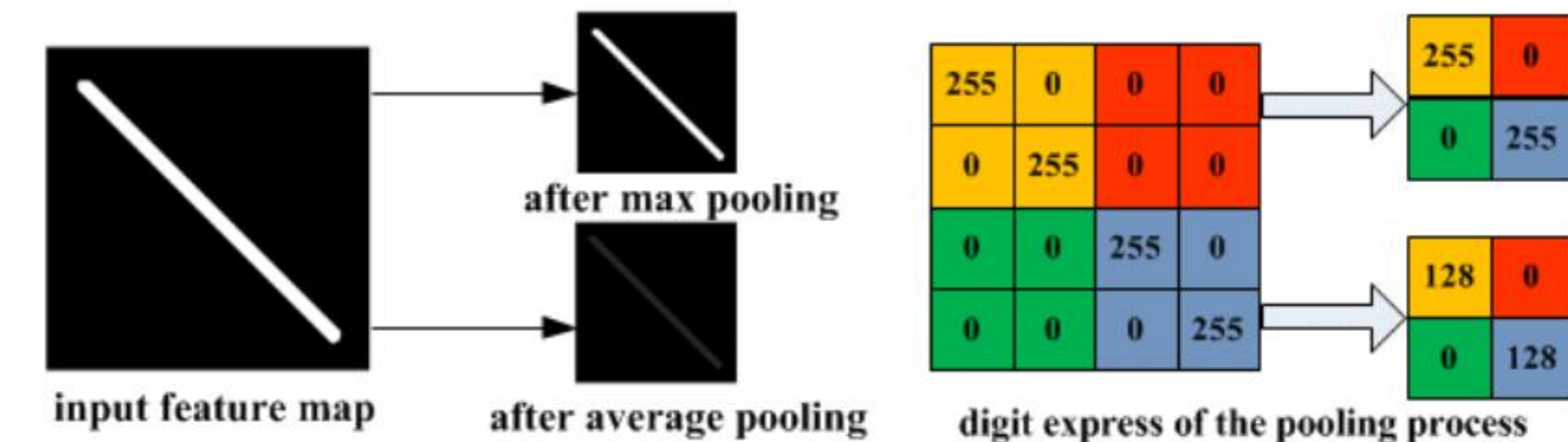
- **Smoothing:** Softens features, creating a more generalized map.
- **Less Common Early On:** It blurs sharp features (like edges), undesirable in early CV stages.



Analogy: Pooling is like reading a paragraph and summarizing the core idea instead of memorizing every word.



(a) Illustration of max pooling drawback



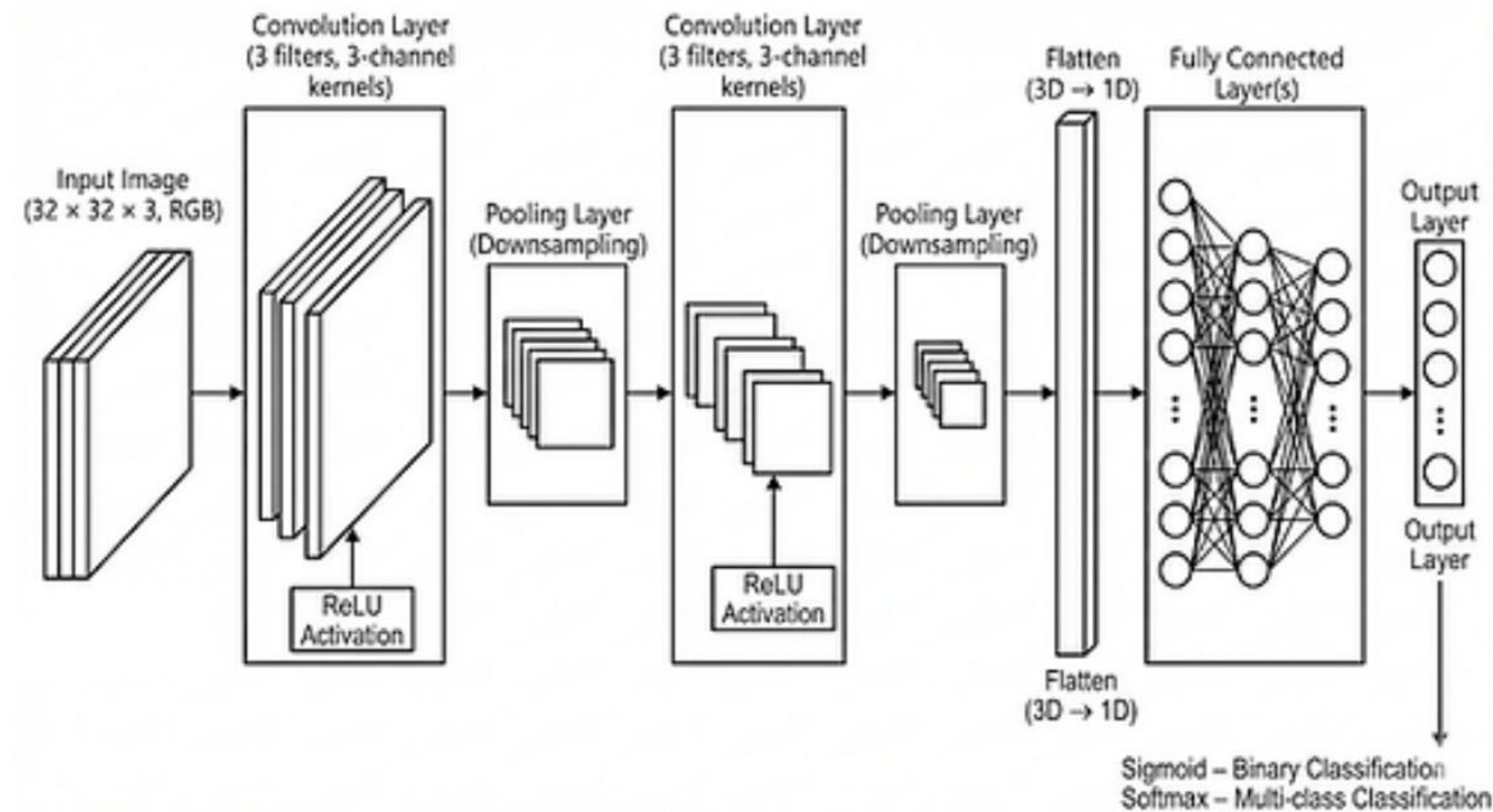
(b) Illustration of average pooling drawback

Pooling Architectures Compared

Pooling Type	Operation	Advantage	Common Application
Max Pooling	Selects max value.	Preserves strongest features (edges).	Intermediate layers (VGG, ResNet).
Average Pooling	Calculates mean value.	Smooths out spatial information.	Historical networks (LeNet).
Global Avg. Pooling (GAP)	Averages entire map into one number.	Reduces parameters, prevents overfitting.	Final layers before classification.

Putting Everything Together: The Pipeline

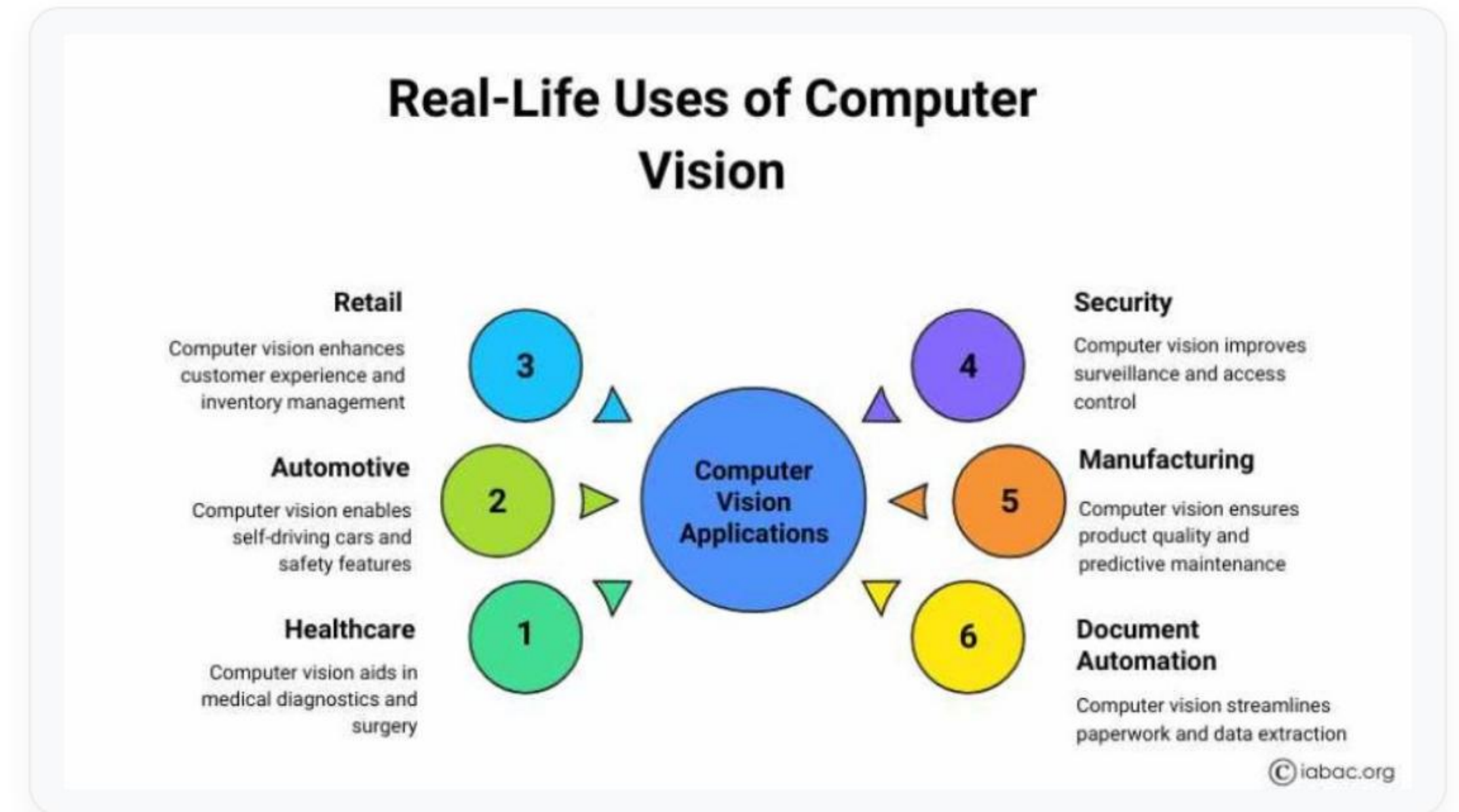
A complete CNN block combines Convolution (with Padding and Stride), Activation (ReLU), and Pooling to form hierarchical feature extractors.



Real-world Applications

Efficient feature extraction (driven by padding, stride, and pooling) is why CNNs dominate real-world tasks:

- **Medical Imaging:** Detecting tumors with localized fields.
- **Self-driving Cars:** Real-time object detection.
- **Face Recognition:** Extracting invariant facial features.
- **Industrial Inspection:** Microscopic defect detection.



Lecture Summary

- ✓ **Padding** preserves border information and dimensions.
- ✓ **Stride** controls how the kernel moves, impacting size.
- ✓ **Output Size** depends on Input, Kernel, Padding, and Stride.
- ✓ **Receptive Fields** grow with depth to capture global context.
- ✓ **Pooling** reduces dimensions, saving memory.

Think–Pair–Share & Homework

In-Class Discussion

Question: If you completely removed all Pooling layers from a deep CNN, what happens to the network?

Discuss with your neighbor for 2 minutes.

Homework Assignment

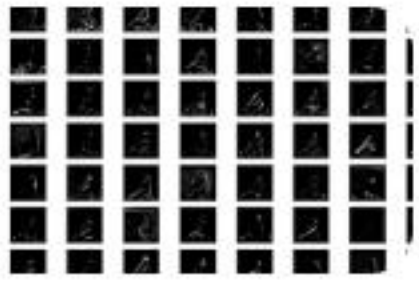
Calculate the Output Size:

- Input = 32×32 , Kernel = 3×3
- Padding = 1, Stride = 2

Short Essay: Explain why Max Pooling is more common than Average Pooling in modern CNNs.

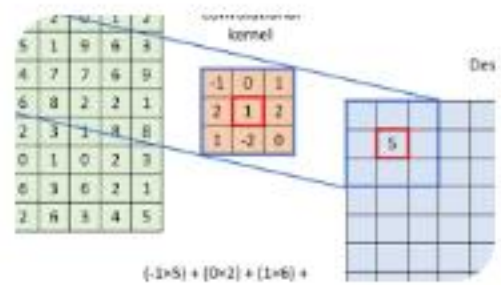
References: Stanford CS231n Notes, Deep Learning (Goodfellow). Next Lecture: Complete CNN Architectures.

Image Sources



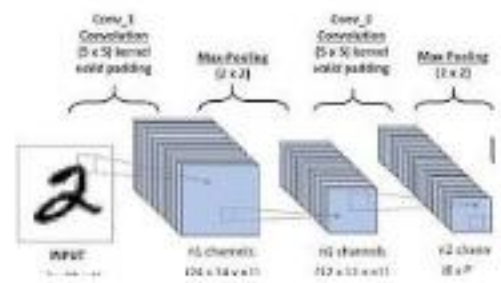
<https://machinelearningmastery.com/wp-content/uploads/2019/02/Visualization-of-the-Feature-Maps-Extracted-from-the-Block-3-in-the-VGG16-Model-1024x768.png>

Source: machinelearningmastery.com



https://miro.medium.com/1*tNQvssqUaiYteDpREHQyFw.png

Source: medium.com



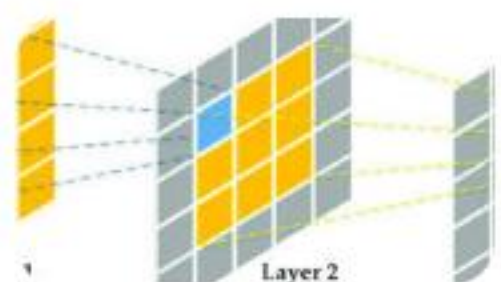
https://miro.medium.com/0*vuja0HL-oTjAZVPW.jpeg

Source: medium.com

$$\begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix} * \begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix} = \begin{bmatrix} 0 & 8 \\ 6 & 8 \end{bmatrix}$$

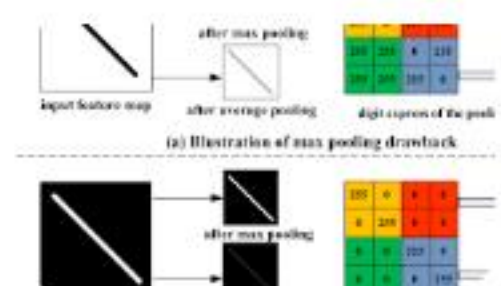
https://d2l.ai/_images/conv-stride.svg

Source: d2l.ai



https://miro.medium.com/1*fNvIU48e8UICdS-luC1duw.png

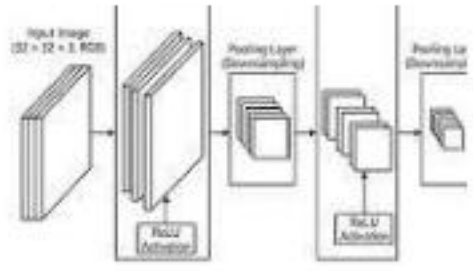
Source: medium.com



https://miro.medium.com/v2/resize:fit:1318/1*ypIfJX7iWX6h6Kbkfq85Kg.png

Source: pub.towardsai.net

Image Sources



https://static.wixstatic.com/media/468fc1_48fc679427b1447ab621c047a38fdbfa~mv2.png/v1/fill/w_568,h_276,al_c,q_85,usm_0.66_1.00_0.01,enc_avif,quality_auto/468fc1_48fc679427b1447ab621c047a38fdbfa~mv2.png

Source: www.aryanupadhyay.com



https://iabac.org/blog/uploads/images/202507/image_870x_688997d42b7ac.jpg

Source: iabac.org